HARI VIGNESH RAO 23BD1A052Q

Experiment – 2

Analysing Android App Permissions and Mobile Traffic

1. Aim

To analyse Android application network traffic and permissions by intercepting HTTP and HTTPS requests using an Android Emulator and Burp Suite.

2. Objective

- To create and run an Android Virtual Device (AVD).
- To configure a proxy between the Android Emulator and Burp Suite.
- To capture HTTP traffic from the Android device.
- To intercept and analyse HTTPS traffic by installing the Burp Suite CA certificate.

3. Required Tools / Software

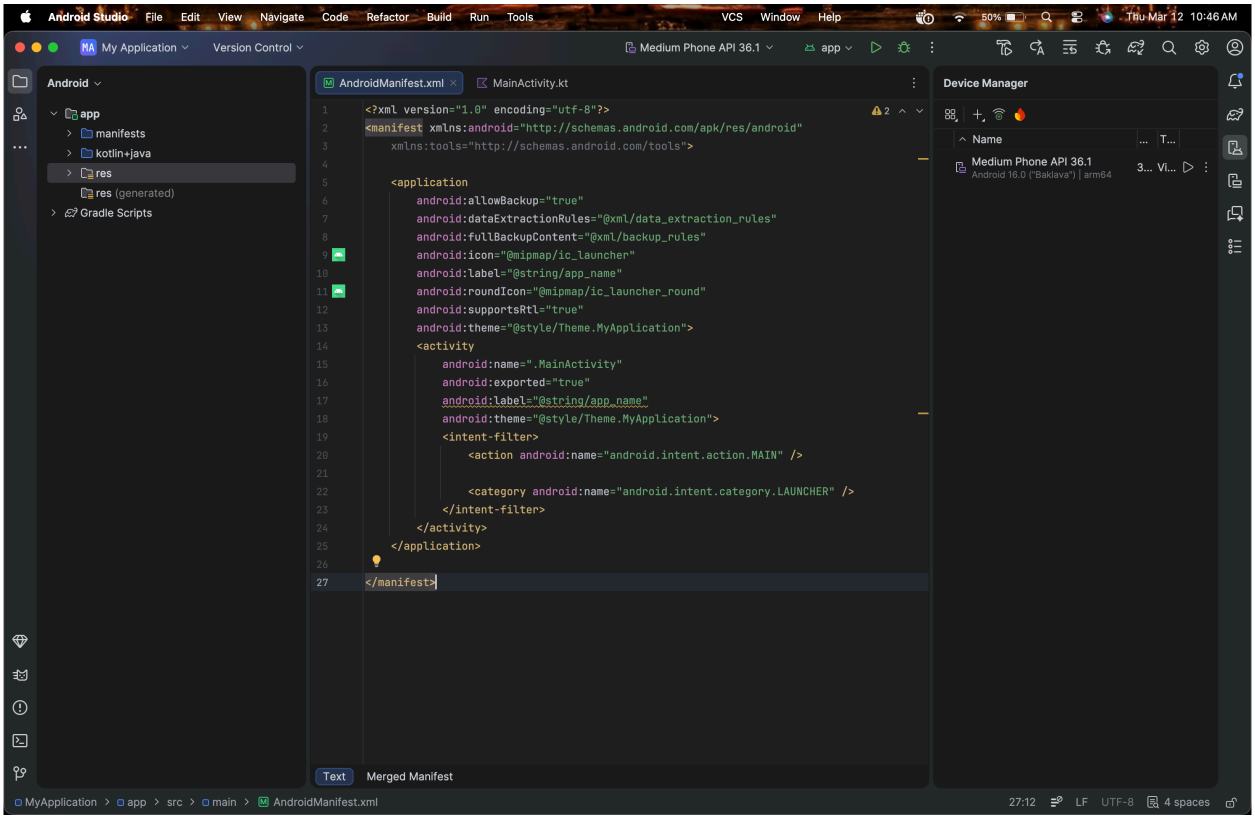
- Android Studio
- Android Emulator (AVD Manager)
- Burp Suite
- ADB (Android Debug Bridge)
- Google Chrome (inside emulator)
- Internet connection

4. Procedure

Step 1: Create Android Virtual Device

1. Install **Android Studio**.
2. Open **AVD Manager**.
3. Create a new Virtual Device.

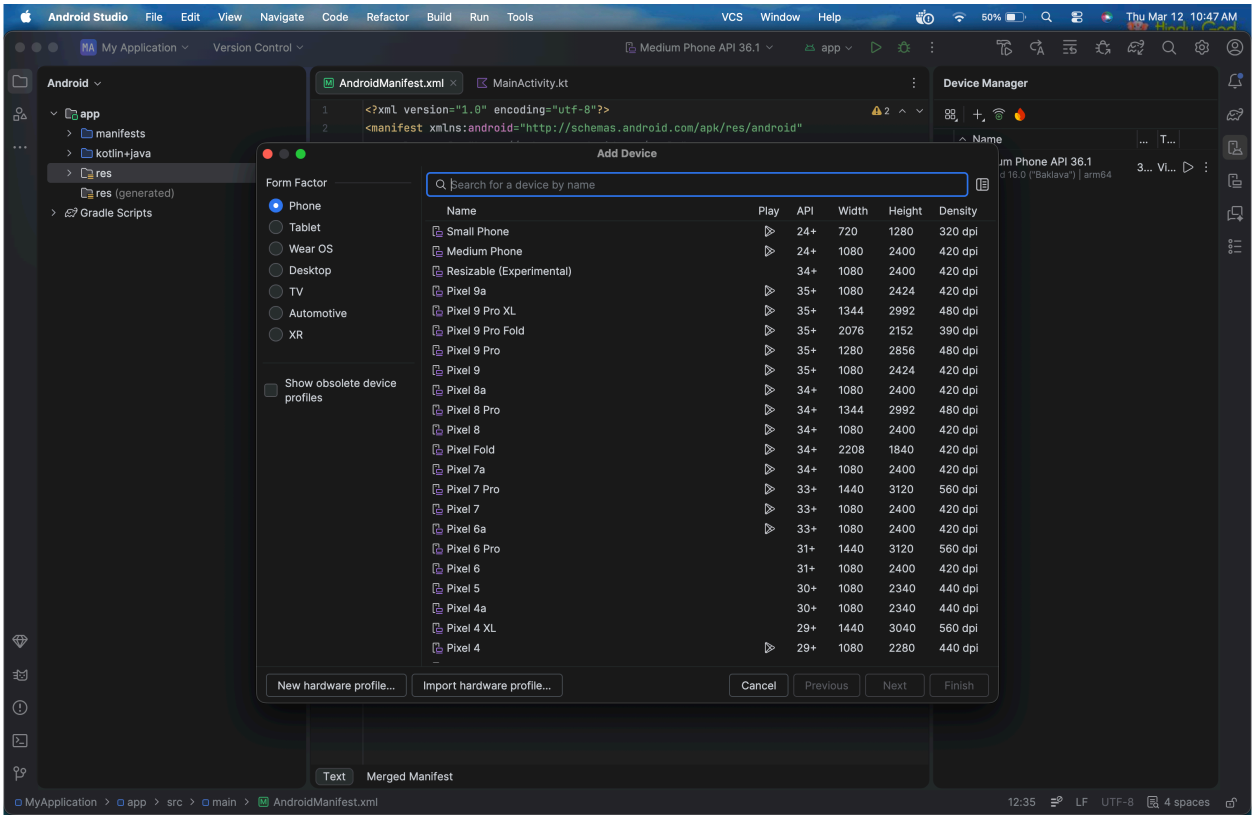
Step-1: Download Android Studio->Create Virtual Device



Step 2: Select Device

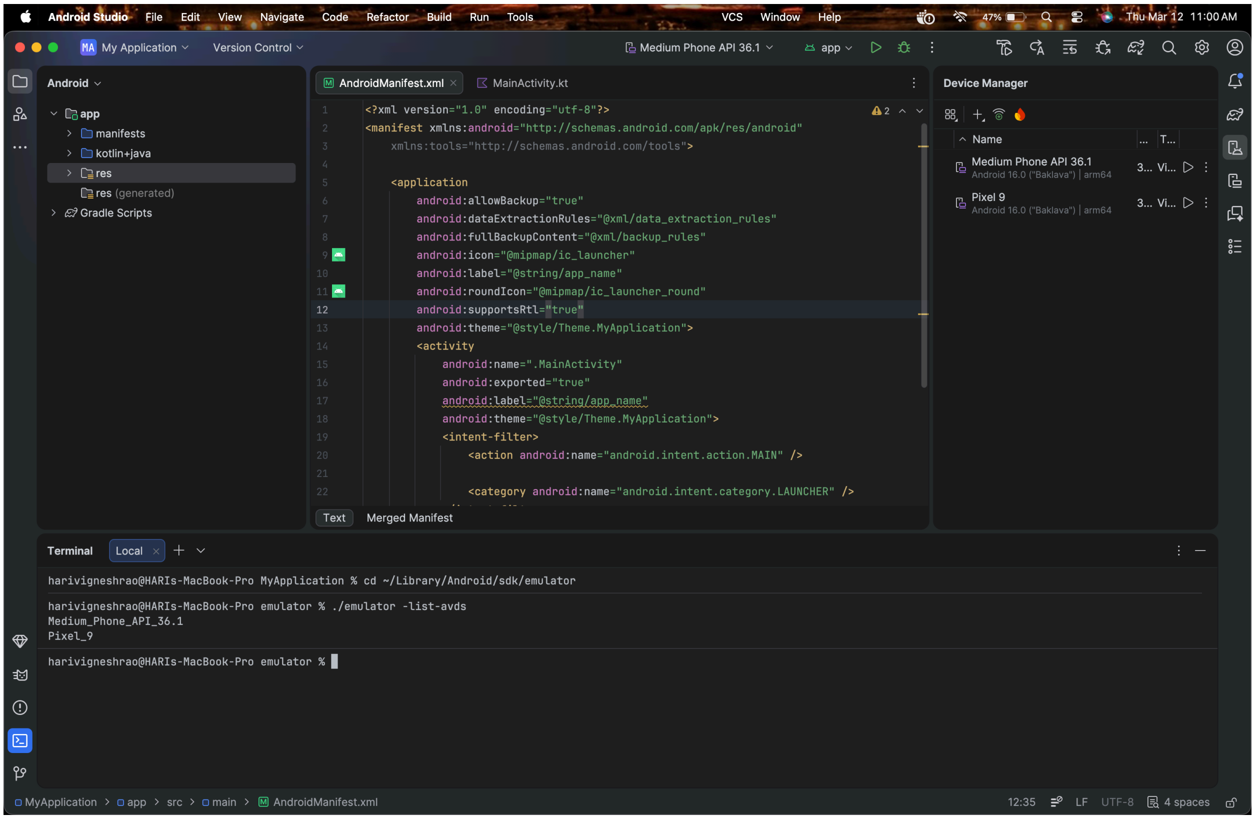
Choose any device such as:

- Pixel 2
- Pixel 9

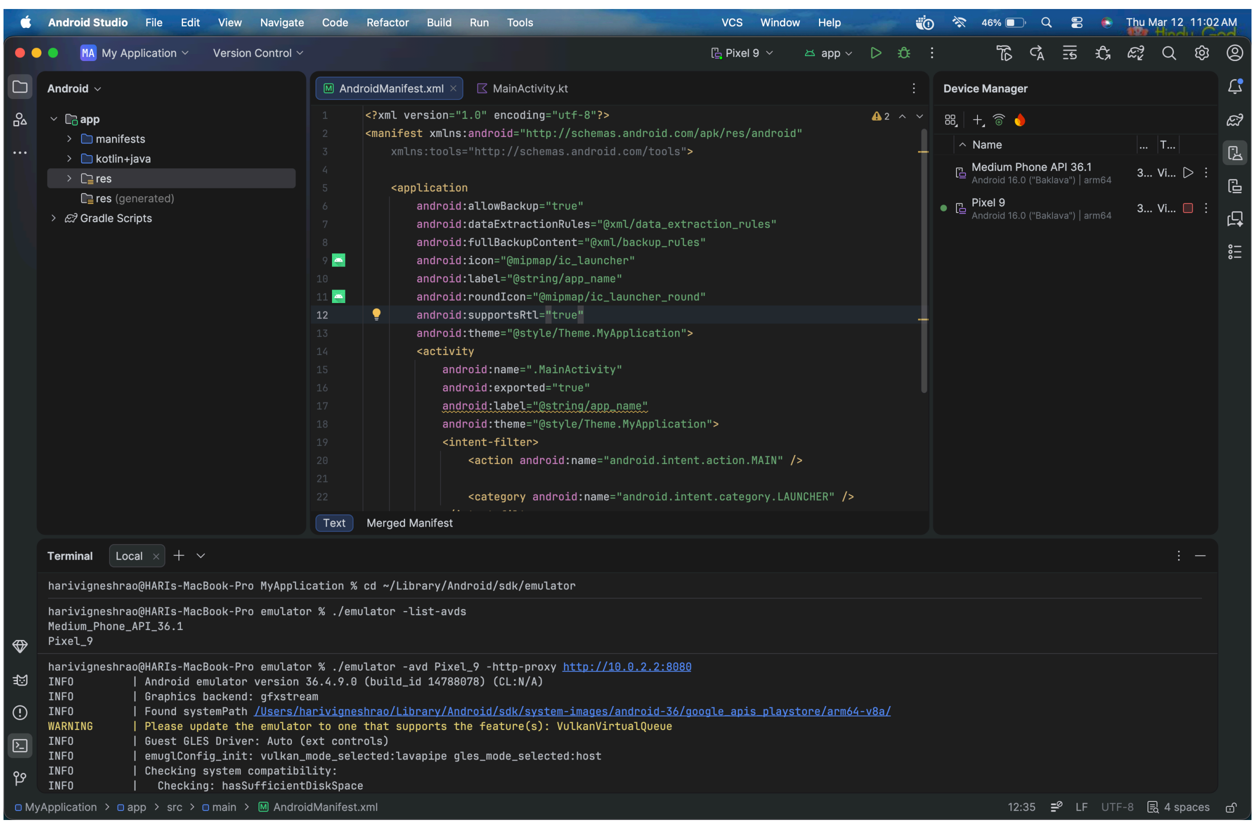


Step-3: Start AVD from Command Prompt1

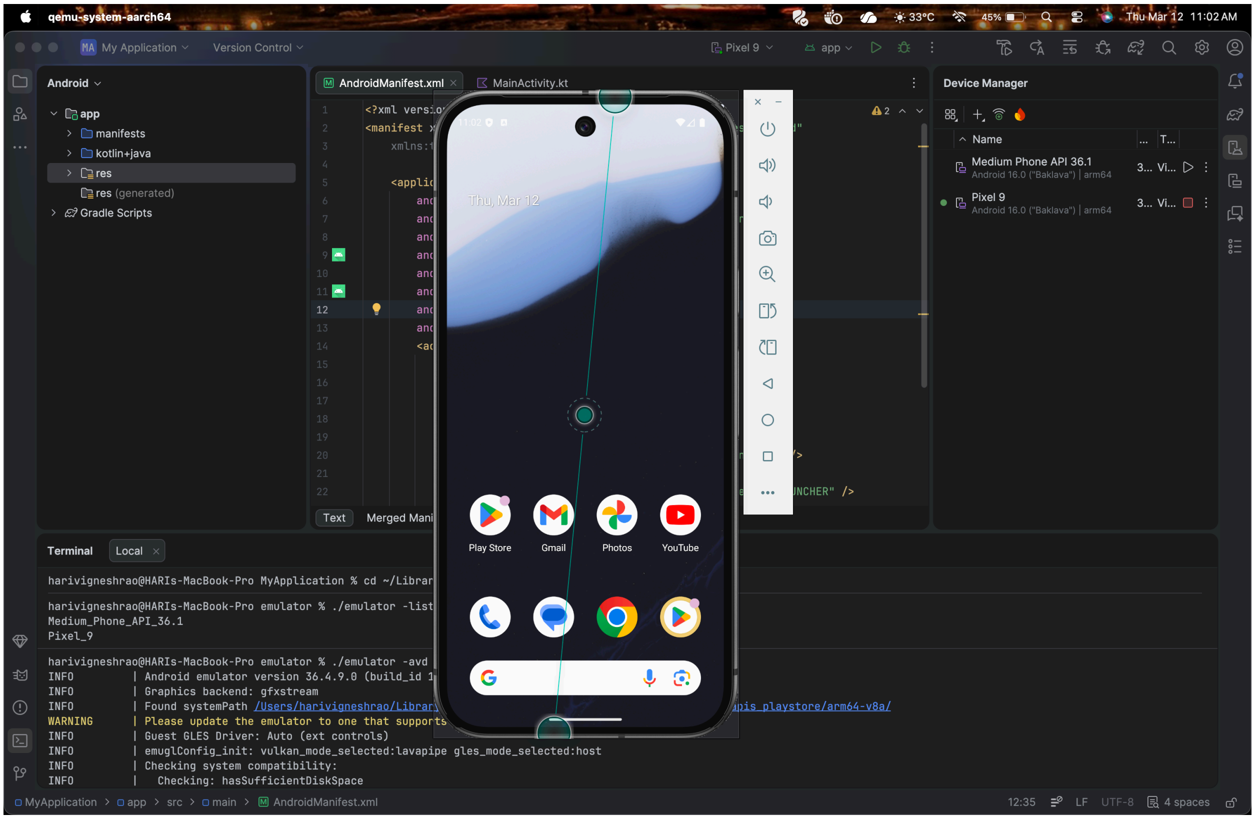
C:\Users\Admin>cd C:\Users\Admin\AppData\Local\Android\Sdk\emulator
C:\Users\Admin\AppData\Local\Android\Sdk\emulator>emulator -list-avds
Medium_Phone_API_36.1
Pixel_2
Pixel_9
C:\Users\Admin\AppData\Local\Android\Sdk\emulator>emulator -avd Pixel_9 -http-proxy http://10.0.2.2:8080



Android Studio File Edit View Navigate Code Refactor Build Run Tools VCS Window Help
Thu Mar 12 11:00 AM
My Application
Version Control
Medium Phone API 36.1
app
Android
app
manifests
kotlin+java
res
res (generated)
Gradle Scripts
AndroidManifest.xml
MainActivity.kt
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools">
<application
android:allowBackup="true"
android:dataExtractionRules="@xml/data_extraction_rules"
android:fullBackupContent="@xml/backup_rules"
android:icon="@mipmap/ic_launcher"
android:label="@string/app_name"
android:roundIcon="@mipmap/ic_launcher_round"
android:supportsRtl="true"
android:theme="@style/Theme.MyApplication">
<activity
android:name=".MainActivity"
android:exported="true"
android:label="@string/app_name"
android:theme="@style/Theme.MyApplication">
<intent-filter>
<action android:name="android.intent.action.MAIN" />
<category android:name="android.intent.category.LAUNCHER" />
Text
Merged Manifest
Device Manager
Name
Medium Phone API 36.1
Android 16.0 ("Baklava") | arm64
Pixel 9
Android 16.0 ("Baklava") | arm64
Terminal
Local
harivigneshrao@HARIs-MacBook-Pro MyApplication % cd ~/Library/Android/sdk/emulator
harivigneshrao@HARIs-MacBook-Pro emulator % ./emulator -list-avds
Medium_Phone_API_36.1
Pixel_9
harivigneshrao@HARIs-MacBook-Pro emulator %
MyApplication > app > src > main > AndroidManifest.xml
12:35 LF UTF-8 4 spaces



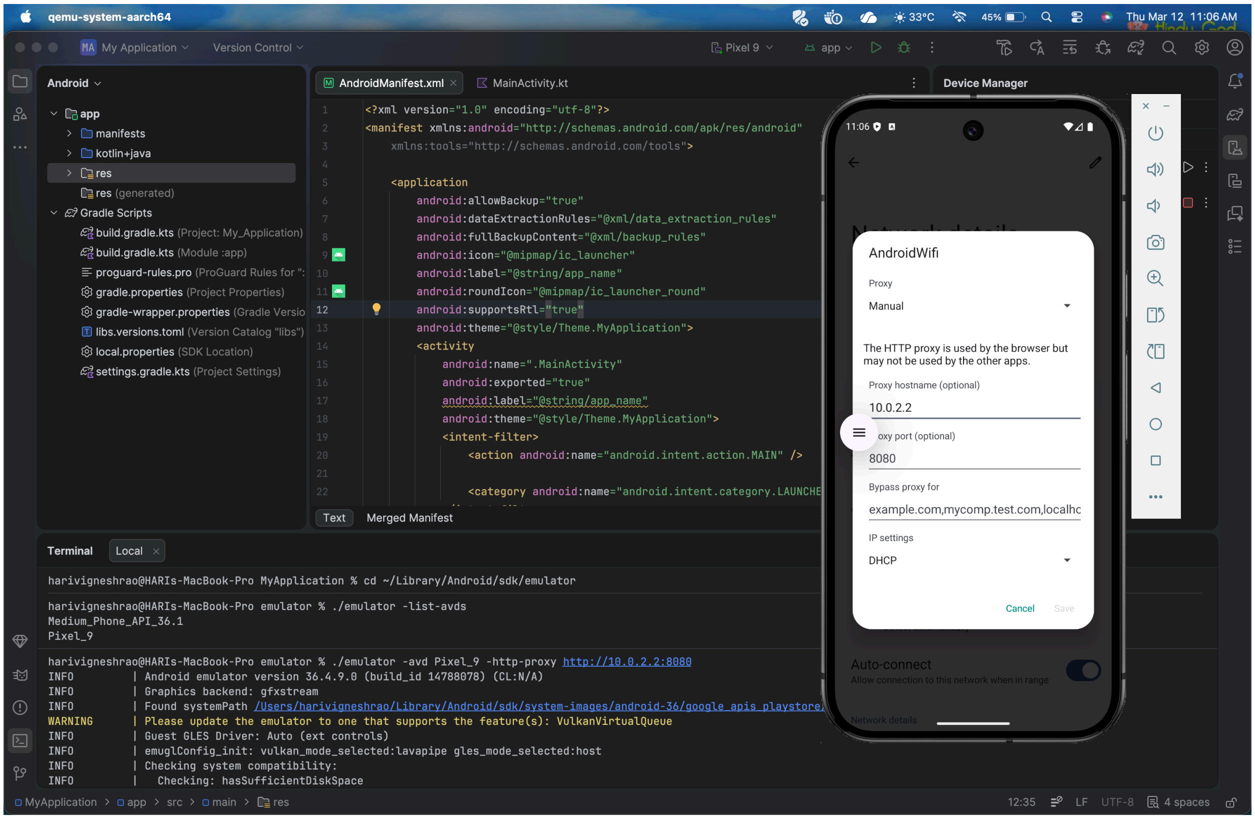
Android Studio File Edit View Navigate Code Refactor Build Run Tools VCS Window Help
Thu Mar 12 11:02 AM
My Application
Version Control
Pixel 9
app
Android
app
manifests
kotlin+java
res
res (generated)
Gradle Scripts
AndroidManifest.xml
MainActivity.kt
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools">
<application
android:allowBackup="true"
android:dataExtractionRules="@xml/data_extraction_rules"
android:fullBackupContent="@xml/backup_rules"
android:icon="@mipmap/ic_launcher"
android:label="@string/app_name"
android:roundIcon="@mipmap/ic_launcher_round"
android:supportsRtl="true"
android:theme="@style/Theme.MyApplication">
<activity
android:name=".MainActivity"
android:exported="true"
android:label="@string/app_name"
android:theme="@style/Theme.MyApplication">
<intent-filter>
<action android:name="android.intent.action.MAIN" />
<category android:name="android.intent.category.LAUNCHER" />
Text
Merged Manifest
Device Manager
Name
Medium Phone API 36.1
Android 16.0 ("Baklava") | arm64
Pixel 9
Android 16.0 ("Baklava") | arm64
Terminal
Local
harivigneshrao@HARIs-MacBook-Pro MyApplication % cd ~/Library/Android/sdk/emulator
harivigneshrao@HARIs-MacBook-Pro emulator % ./emulator -list-avds
Medium_Phone_API_36.1
Pixel_9
harivigneshrao@HARIs-MacBook-Pro emulator % ./emulator -avd Pixel_9 -http-proxy http://10.0.2.2:8080
INFO | Android emulator version 36.4.9.0 (build_id 14788078) (CL:N/A)
INFO | Graphics backend: gfxstream
INFO | Found systemPath /Users/harivigneshrao/Library/Android/sdk/system-images/android-36/google_apis_playstore/arm64-v8a/
WARNING | Please update the emulator to one that supports the feature(s): VulkanVirtualQueue
INFO | Guest GLES Driver: Auto (ext controls)
INFO | emuglConfig_init: vulkan_mode_selected:lavapipe gles_mode_selected:host
INFO | Checking system compatibility:
INFO | Checking: hasSufficientDiskSpace
MyApplication > app > src > main > AndroidManifest.xml
12:35 LF UTF-8 4 spaces



Step-4: Setup Proxy:

In AVD

Settings-> Network and Internet-> internet->select icon -> top right corner there is edit->advanced options -> proxy->Change proxy None->Manual->Hostname as 10.0.2.2 ->save



Note: if save option is not working then follow the steps in command prompt

Command Prompt2:

C:\Users\Admin>cd C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb devices

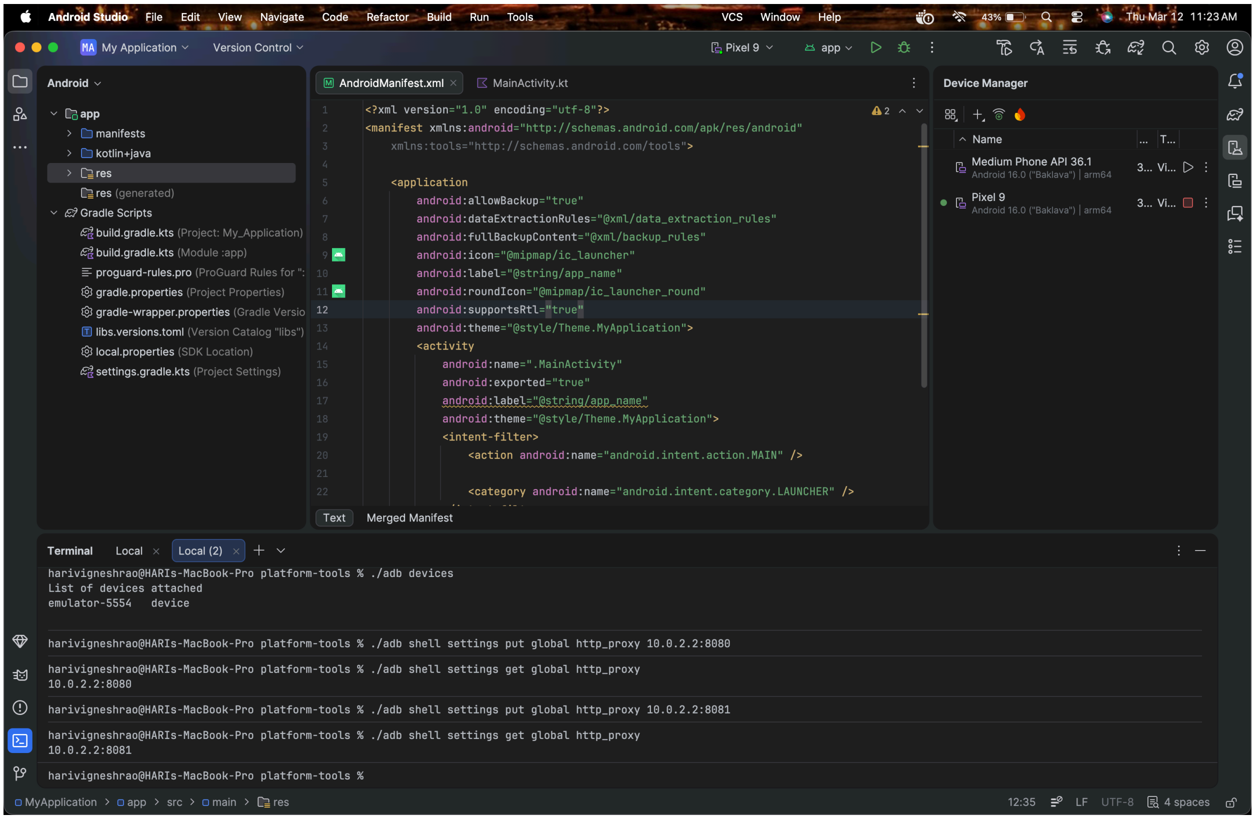
List of devices attached

emulator-5554 device

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb shell settings put global http_proxy 10.0.2.2:8080

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb shell settings get global http_proxy

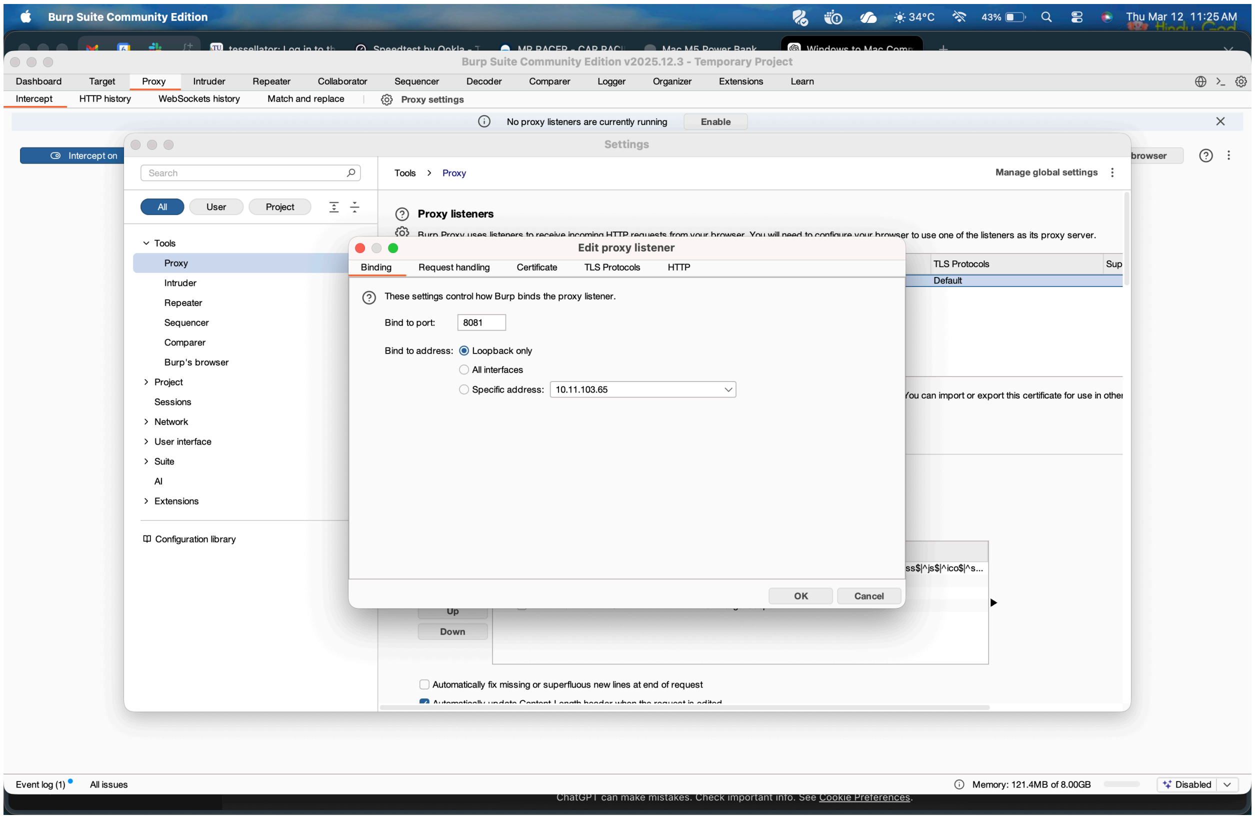
10.0.2.2:8080



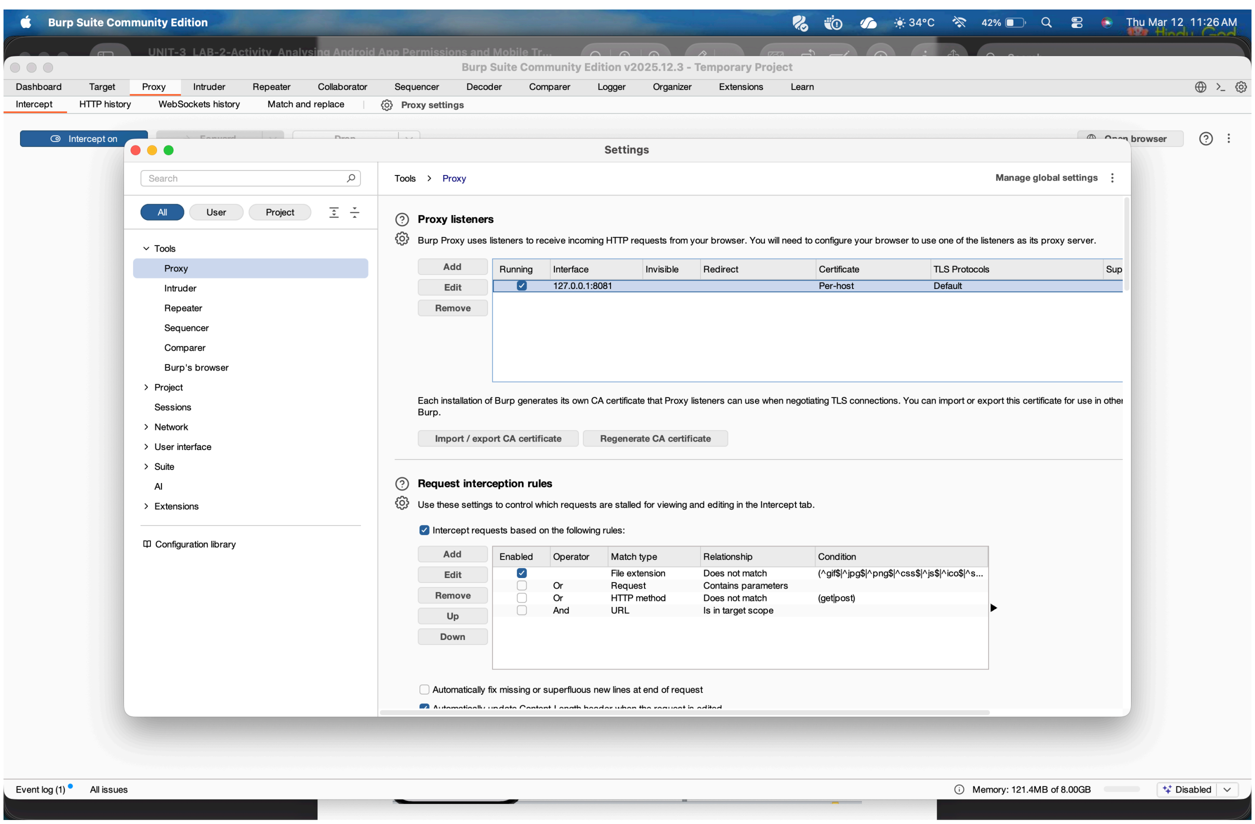
Step 5: In BurpSuit

Go procy->options/proxy settings->under Proxy Listeners->Edit->select All interfaces->ok

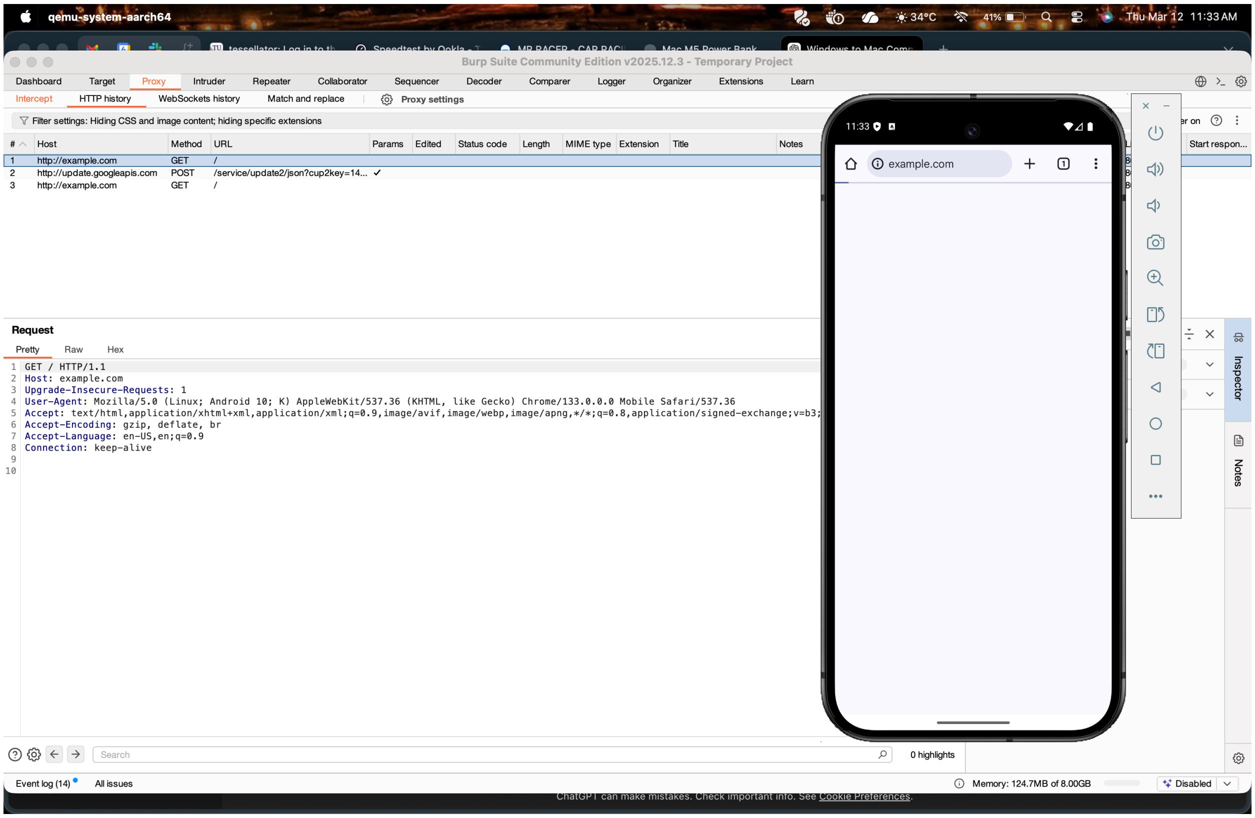
Go proxy->intercept on



You can see BurpSuit is accepting all interfaces traffic

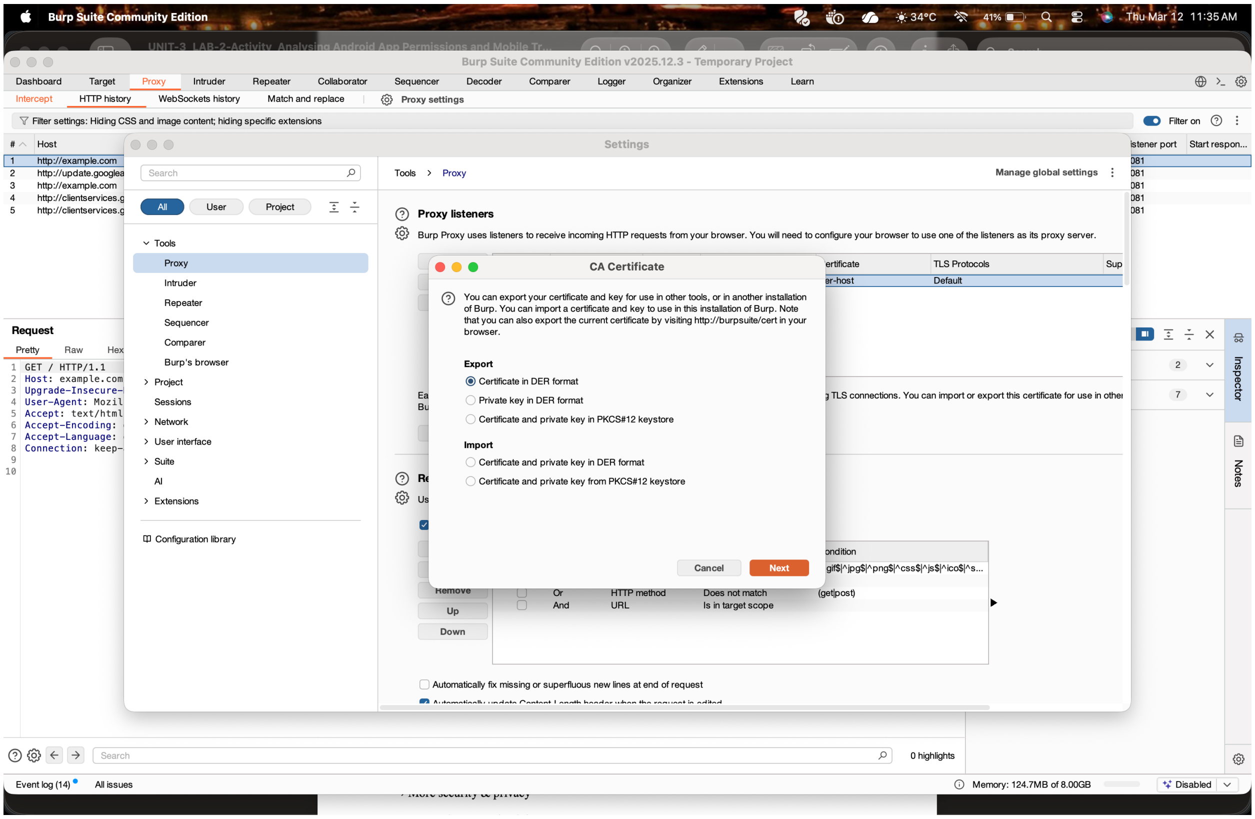


Step 6: in AVD type in chrome example.com and you can see burpsuit HTTP history



Step 7: in Step 5 we only able to http request and response only but if you want to analyse https request and response follow the Procedure to download CA certificate from BunrpSuit.

Go procy->options/proxy settings->import/export CA certificate->Certificate in DER format->select file-> Downloads->give name as burpcer.der

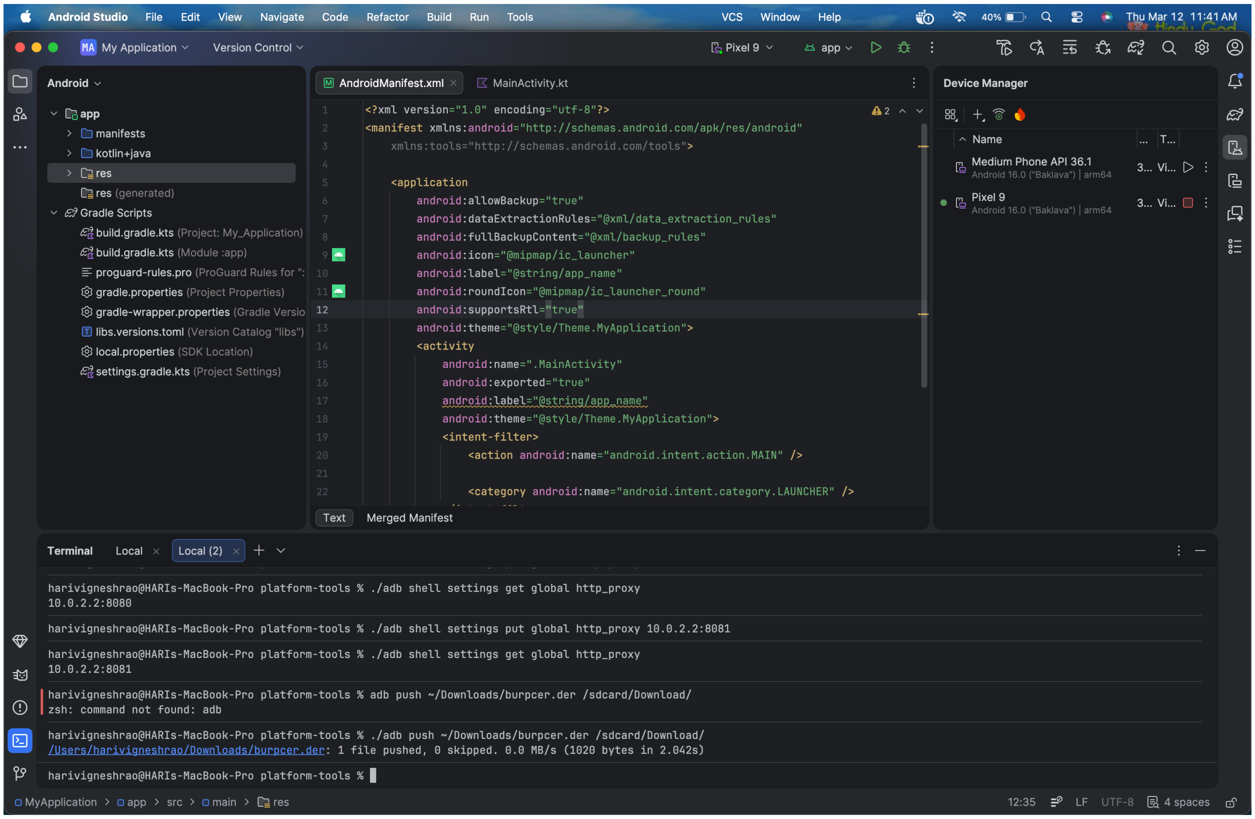


Step 8: Sending CA certificate to AVD

Go to command Prompt2

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb push

C:\Users\Admin\Downloads\burpcer.cer /sdcard/Download/



To install Certificate in AVD

Settings

→ Security & privacy

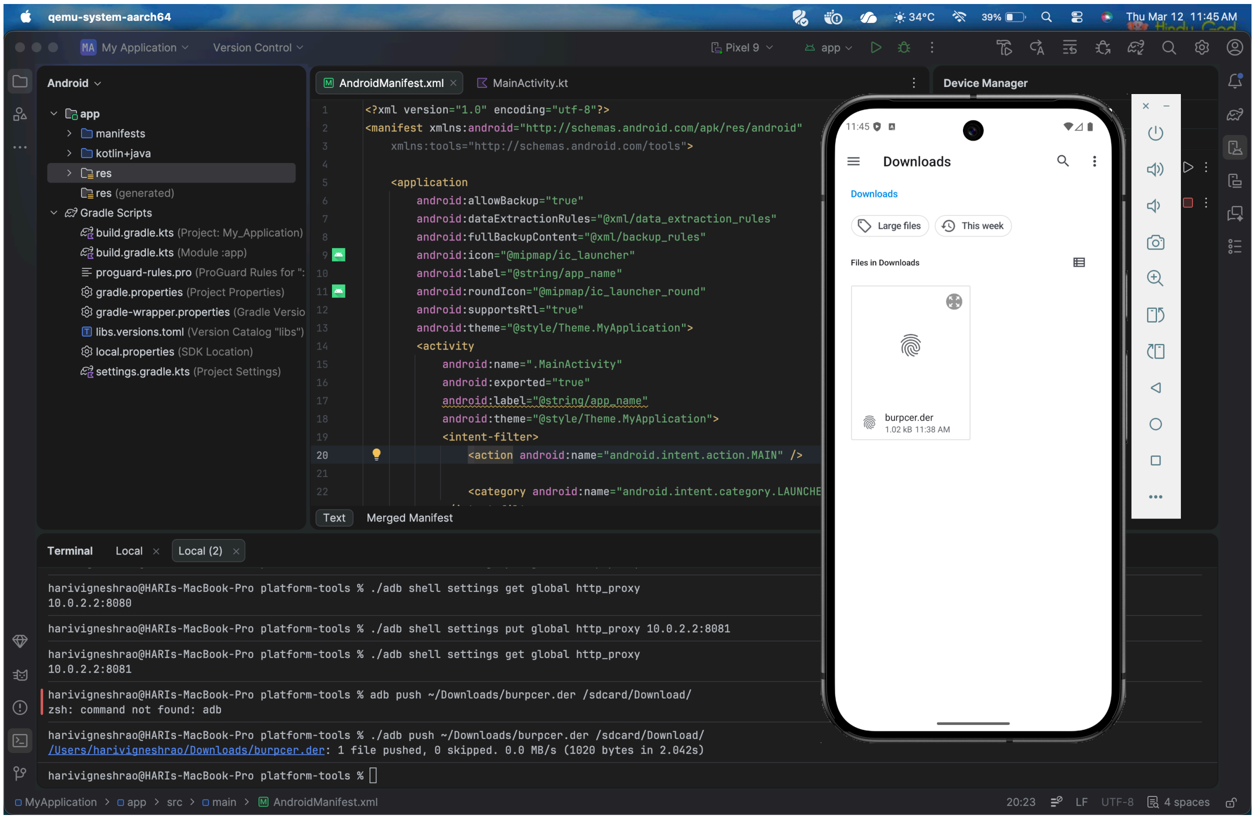
→ More security & privacy

→ Encryption & credentials

→ Install a certificate

→ CA certificate

In AVD we can see burpcer.der certificate

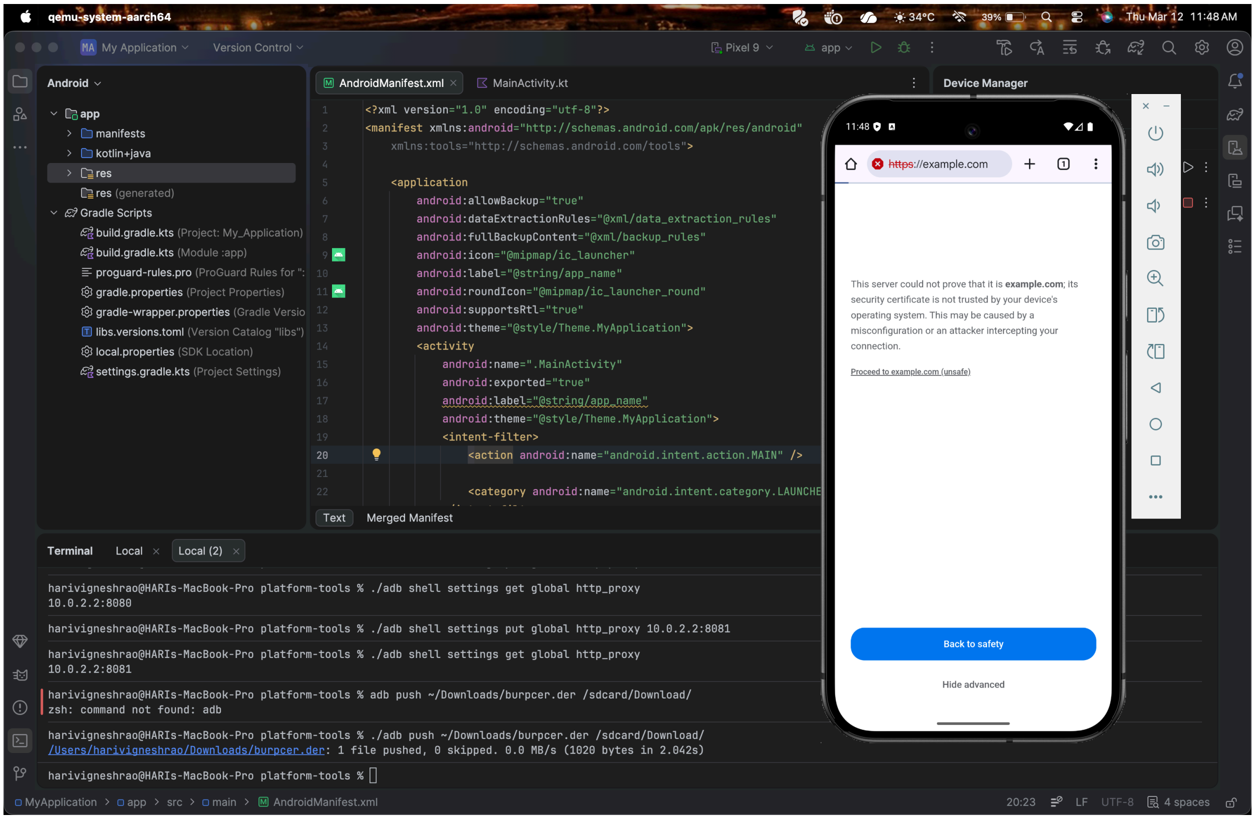


Step 9:verify HTTPS Interception

1. Open browser in emulator.
2. Visit any HTTPS website.

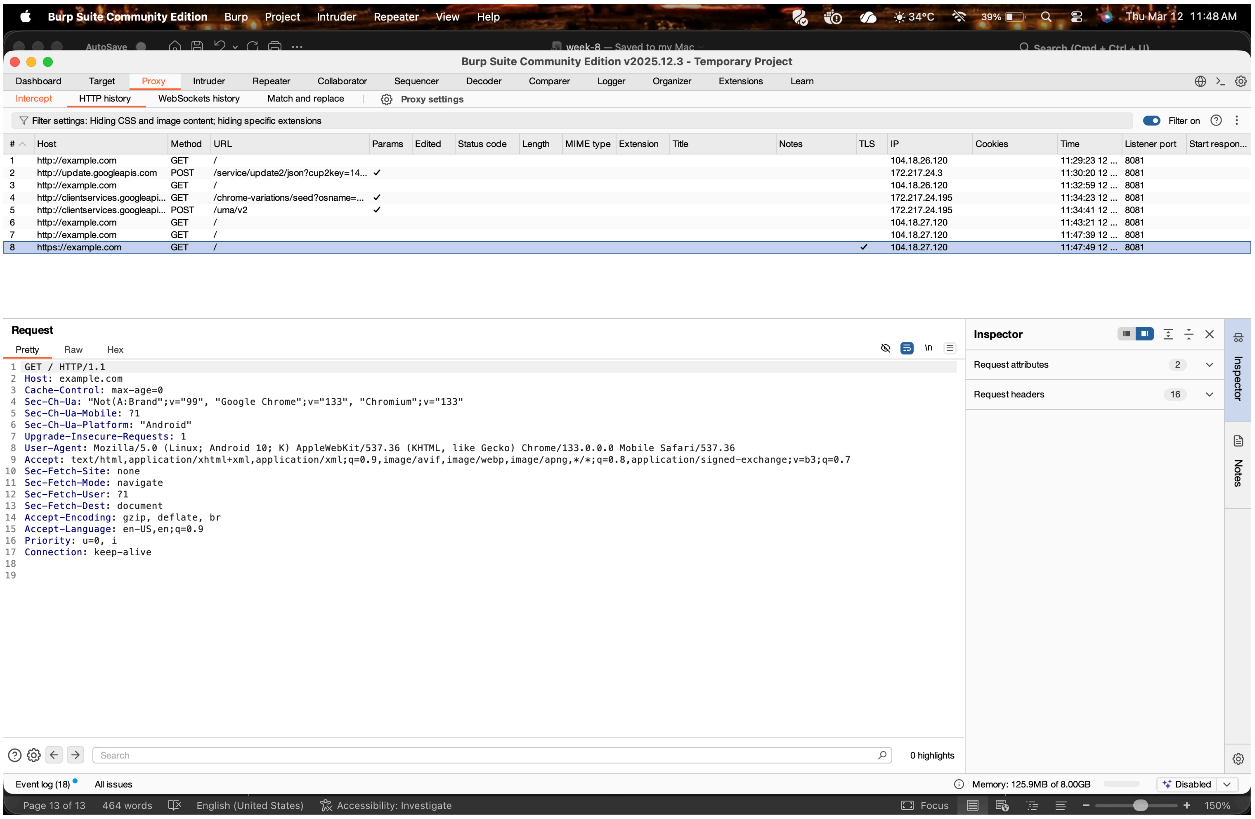
Example:

`https://example.com` or https://wikipedia.com



3. In Burp Suite, check **Proxy → HTTP History**.

Now HTTPS requests and responses will be visible.



5. Result

Android application network traffic was successfully intercepted and analysed using Burp Suite by configuring proxy settings in the Android emulator and installing the Burp Suite CA certificate.

6. Precautions

• Ensure Burp Suite proxy port (8080) is correctly configured.
• Verify the emulator is running before executing ADB commands.
• Install the CA certificate properly to capture HTTPS traffic.• Disable intercept when not required to allow traffic to pass normally.